IBM Innovation Centre for Education

NCS Nihon Communication Solutions

YENEPOYA
(DEEMED TO BE UNIVERSITY)

YENEPOYA UNIVERSITY

Final Project Report

ON

IDENTITY SECURITY FRAMEWORK FOR SMART CITY

Student Name: Ganga N S

Industry Mentor: Mr . Shashank Korram **Guided by:**Vikyath Shetty

TABLE OF CONTENTS

Section No.	Heading	Page No.
1	Background	5
1.1	Aim	5
1.2	Technologies	6
1.3	Hardware Architecture	7
1.4	Software Architecture	8
2	System	9
2.1	Requirements	9
2.1.1	Functional Requirements	10
2.1.2	User Requirements	10
2.1.3	Environmental Requirements	11
2.2	Design and Architecture	11
2.3	Implementation	15
2.4	Testing	15
2.5	Graphical User Interface (GUI) Layout	16
2.6	Customer Testing	16
2.7	Evaluation	16
3	Snapshots of the Project	18
4	Conclusions	21
5	Further Development or Research	22
6	References	23
7	Appendix	24

EXECUTIVE SUMMARY

The project titled “Identity Security Framework for Smart City” focuses on developing a secure and efficient authentication system using Python, Flask, and SQLite. The system is designed to manage user identity securely and provide controlled access to applications in a Smart City environment.

Overview

The application is a web-based system that allows users to register, log in, and log out securely. It follows a client-server architecture where the user interacts through a web browser, and the backend processes requests using Flask.

Objective

The main objective of the project is to implement a secure authentication mechanism that protects user data. It emphasizes concepts such as password hashing, session management, and user validation.

Key Features

- User registration and login system
- Secure password storage using hashing
- Session management using Flask-Login
- Input validation and error handling

Outcome

The system successfully demonstrates a basic identity security framework with efficient performance and user-friendly design. It ensures secure handling of user data and meets the project requirements.

Future Scope

The project can be further enhanced by adding features such as multi-factor authentication, admin panel, and cloud deployment to support large-scale Smart City application

1. BACKGROUND

Smart Cities use digital technologies to improve services such as transportation, healthcare, and governance. As more services become available online, users need to authenticate themselves to access these systems. This makes identity security an important part of any Smart City application.

In many traditional systems, authentication methods are not secure enough. Passwords may be stored improperly, and weak validation can lead to vulnerabilities like unauthorized access or data breaches. These issues highlight the need for secure authentication mechanisms that protect user information.

This project focuses on building a simple and secure login system using modern practices such as password hashing and session management. By using technologies like Flask and SQLite, the system ensures that user credentials are handled safely and efficiently. The project also demonstrates key cybersecurity concepts such as authentication and authorization.

Overall, this system provides a basic but effective approach to identity security, which can be further expanded for real-world Smart City applications.

1.1 Aim

The aim of this project is to design and develop a secure and efficient identity authentication system for Smart City applications using Python, Flask, and SQLite. The system is intended to provide a simple and reliable platform where users can register, log in, and access services in a secure manner.

This project focuses on implementing fundamental cybersecurity practices such as password hashing, input validation, and secure session management to ensure that user credentials are protected from unauthorized access. It also aims to demonstrate the difference between authentication (verifying user identity) and authorization (controlling user access to resources) in a practical environment.

Another important aim of this project is to provide hands-on experience in developing web-based applications using modern frameworks like Flask. It helps in understanding how backend systems interact with databases and how secure data handling is achieved in real-world scenarios.

Furthermore, the project is designed to be lightweight and scalable, so that it can serve as a foundation for future enhancements such as role-based access control, multi-factor authentication, and cloud-based deployment. Overall, the aim is to build a system that not only ensures security but also improves usability and efficiency in Smart City digital services.

1.2 Technology

This project uses a combination of programming languages, frameworks, and tools to build a secure and efficient authentication system. Each technology is selected based on its simplicity, performance, and suitability for developing web-based applications.

Python is used as the primary programming language for developing the backend of the application. It is widely known for its simplicity, readability, and strong community support. Python allows developers to quickly build applications with fewer lines of code, making it an ideal choice for implementing web-based systems and security features.

Flask is a lightweight web framework used to develop the server-side logic of the application. It provides essential features such as routing, request handling, and template rendering. Flask is highly flexible and allows developers to build applications without unnecessary complexity, making it suitable for small to medium-scale projects like this one.

Flask-Login is an extension of Flask that is specifically used for managing user authentication and sessions. It simplifies the process of handling user login, logout, and session tracking. It also provides features such as restricting access to certain pages unless the user is authenticated, thereby improving the overall security of the application.

SQLite is used as the database for storing user information such as email and password hashes. It is a lightweight, file-based database that does not require a separate server to run. SQLite is easy to set up and is well-suited for small-scale applications and learning purposes.

HTML and CSS are used for designing the frontend of the application. HTML is used to create the structure of web pages such as login and signup forms, while CSS is used to style these pages and improve user experience. Together, they provide a simple and user-friendly interface.

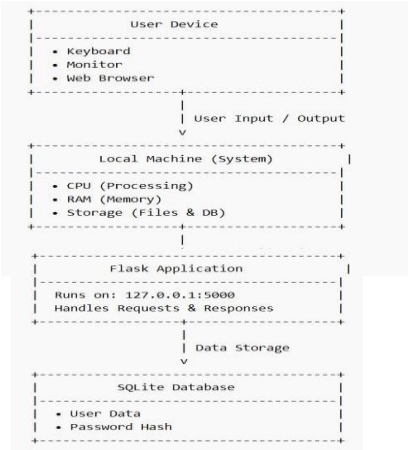
Werkzeug Security (built-in with Flask) is used for password hashing and verification. It ensures that passwords are not stored in plain text but are converted into secure hashed values, making it difficult for attackers to retrieve the original passwords even if the database is compromised.

Visual Studio Code (VS Code) is used as the development environment for writing and managing the project code. It provides features such as syntax highlighting, debugging tools, and extensions that make development easier and more efficient.

Web Browser is used to run and test the application locally. The application is hosted on http://127.0.0.1:5000, where users can interact with the system through the browser interface.

1.3 Hardware Architecture

The hardware architecture of the system is simple and designed to run on a standard personal computer without requiring advanced resources. The application operates on a single-node setup where both the Flask server and SQLite database run on the same machine. The system requires basic hardware components such as a processor (Intel i3 or higher), minimum 4GB RAM, and sufficient storage for project files and database. Users interact with the system through input devices like a keyboard and monitor using a web browser, while the CPU handles processing and RAM manages active operations. This setup ensures efficient performance and easy deployment for small-scale applications.



1.4 Software Architecture

The software architecture of the system follows a three-tier structure consisting of the presen tation layer, application layer, and data layer. The presentation layer, developed using HTML and CSS, provides the user interface for login and signup. The application layer, built using Flask, handles request processing, authentication, and session management with the help of Flask-Login. The data layer uses SQLite to securely store user information such as email and password hash. The layers interact smoothly to process user inputs and generate responses, while security is maintained through password hashing, input validation, and controlled access.

```
+-----------------------------------+
|        Presentation Layer         |
|-----------------------------------|
|  • Login Page (HTML/CSS)          |
|  • Signup Page (HTML/CSS)         |
|  • User Interface                 |
+---------------+-------------------+
                |
                | HTTP Request/Response
                v
+-----------------------------------+
|        Application Layer          |
|-----------------------------------|
|  • Flask (app.py)                 |
|  • Routing (/login, /signup)      |
|  • Authentication Logic           |
|  • Input Validation               |
|  • Flask-Login (Sessions)         |
|  • Password Hashing (Werkzeug)    |
+---------------+-------------------+
                |
                | SQL Queries
                v
+-----------------------------------+
|            Data Layer             |
|-----------------------------------|
|  • SQLite Database                |
|  • User Table                     |
|  • Password Hash Storage          |
+-----------------------------------+
```

2.SYSTEM

The system is a web-based login application that allows users to register and log in securely. It operates on a client-server model where the browser sends requests to the Flask server.
During signup, user data is validated and stored securely in the database after hashing the password. During login, credentials are verified, and a session is created for the user.
The system ensures that only authenticated users can access protected areas, and logout functionality removes session data to maintain security.

2.1Requirements

The requirements of the system define the necessary functionalities, user expectations, and environment needed for the proper working of the application. These requirements ensure that the system performs efficiently and meets user needs while maintaining security and usability.

2.1.1 Functional Requirements

Functional requirements describe what the system is expected to do. These are the core features of the application:

- **User Registration**
  The system should allow new users to create an account by providing basic details such as email and password. The system must validate the input and prevent duplicate registrations.
- **User Login**
  Registered users should be able to log in using their credentials. The system must verify the entered password with the stored hashed password.
- **Password Hashing**
  The system should securely hash user passwords before storing them in the database to ensure data protection.

- **Session Management**

After successful login, the system should create and maintain a user session using Flask-Login. The session should remain active until the user logs out.

- **Logout Functionality**

  The system should allow users to log out, which will terminate the active session and restrict further access.

- **Input Validation**

  The system should validate all user inputs to ensure correctness and prevent malicious data entry.

- **Error Handling**

  Appropriate error messages should be displayed for invalid login attempts, missing fields, or duplicate accounts.

2.1.2 User Requirements

User requirements define what users expect from the system in terms of usability and performance:

- **Simple and User-Friendly Interface**

  The system should have an easy-to-use interface that allows users to navigate without confusion.

- **Fast Response Time**

  The system should process user requests quickly to provide a smooth experience.

- **Secure Access**

  Users expect their data to be protected and accessible only to authorized individuals.

- **Clear Feedback**

  The system should provide clear messages for successful actions and errors.

- **Accessibility**

  The system should be accessible through standard web browsers without requiring special software.

2.1.3 Environmental Requirements

Environmental requirements specify the technical setup needed to run the system:

- **Operating System**
  The system should run on common operating systems such as Windows, Linux, or macOS.
- **Programming Environment**
  Python must be installed along with required libraries such as Flask and Flask-Login.
- **Development Tools**
  Tools like Visual Studio Code can be used for coding and debugging.
- **Database**
  SQLite is required for storing user data.
- **Web Browser**
  A modern browser such as Chrome, Firefox, or Edge is required to access the application.
- **Local Server**
  The application runs on a local server (127.0.0.1:5000).

2.1.4 Non-Functional Requirements

Non-functional requirements describe how the system performs rather than what it does:

- **Security**
  The system must ensure secure storage of passwords using hashing and protect against unauthorized access.
- **Performance**
  The application should respond quickly and handle user requests efficiently.
- **Reliability**
  The system should work consistently without failures during normal usage.
- **Scalability**
  The system should be designed in a way that allows future expansion and additional features.
- **Maintainability**
  The code should be organized and easy to update or modify.
- **Usability**

The interface should be simple and intuitive for all users.

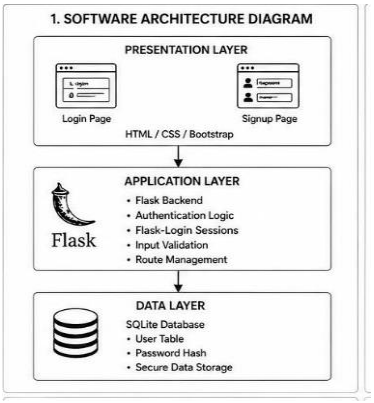
2.2 Design and Architecture

The design and architecture of the system define how different components are structured and how they interact with each other to perform authentication securely. The system is designed using a modular and layered approach to ensure clarity, maintainability, and scalability. It follows standard web application design principles, separating the user interface, application logic, and data storage into distinct layers.

- **System Architecture Design**

The system follows a **three-tier architecture** consisting of the Presentation Layer, Application Layer, and Data Layer. This design ensures that each layer has a specific responsibility, making the system more organized and easier to maintain.

- The **Presentation Layer** handles user interaction through web pages such as login and signup forms.
- The **Application Layer** processes user requests, performs authentication, and manages sessions using Flask and Flask-Login.
- The **Data Layer** stores user information securely in the SQLite database.



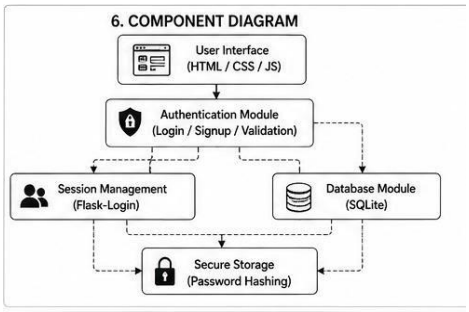
The diagram shows how the three layers interact with each other. The user sends requests through the presentation layer, which are processed by the application layer. The application layer communicates with the database to store or retrieve data and then sends the response back to the user. This structured flow ensures efficient data handling and improved security.

- **Component Design**

The system is divided into different components or modules, each responsible for a specific task. This modular design helps in improving system organization and simplifies debugging and maintenance.

Main components include:

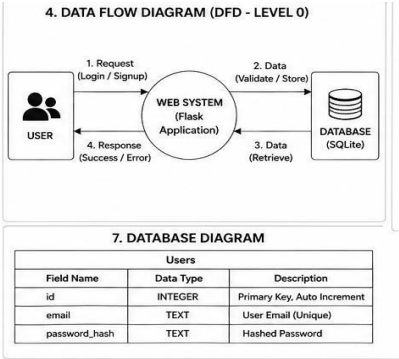
- Authentication module
- Session management module
- Database module
- User interface module



The component diagram illustrates how different modules of the system interact. The user interface communicates with the authentication module, which processes login and signup requests. The authentication module interacts with the database module to verify or store user data. The session module manages user sessions and controls access to protected pages.

- **Data Flow Design**

The data flow design explains how data moves through the system during operations such as login and signup. It focuses on how user input is processed and how responses are generated.



7. DATABASE DIAGRAM

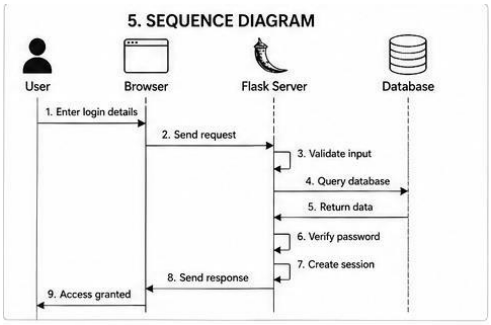
Users		
Field Name	Data Type	Description
id	INTEGER	Primary Key, Auto Increment
email	TEXT	User Email (Unique)
password_hash	TEXT	Hashed Password

The data flow diagram shows the movement of data between the user, application, and database. When a user enters credentials, the data flows from the user interface to the

application layer, where it is validated. The application then communicates with the database to verify the data. The result is sent back to the user as a response. This flow ensures secure and efficient data processing.

- **Sequence Design**

The sequence design describes the step-by-step interaction between system components during a specific operation, such as user login.



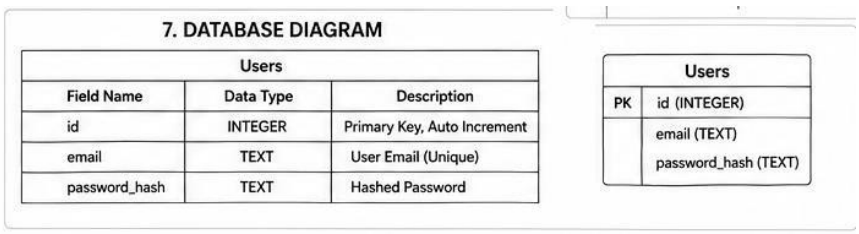
The sequence diagram shows the order of interactions between the user, web interface, Flask server, and database. It begins with the user submitting login details, followed by validation in the application layer. The system retrieves stored data from the database, verifies the credentials, and then creates a session if the login is successful. This step-by-step interaction ensures clarity in system behavior.

- **Database Design**

The database design defines how user data is structured and stored in the system. SQLite is used as the database, and it contains a user table with essential fields.

Table Structure Field Name

Description id Unique user identifier email User email address password_hash Hashed password



7. DATABASE DIAGRAM

Users		
Field Name	Data Type	Description
id	INTEGER	Primary Key, Auto Increment
email	TEXT	User Email (Unique)
password_hash	TEXT	Hashed Password

Users	
PK	id (INTEGER)
	email (TEXT)
	password_hash (TEXT)

The database diagram shows the structure of the user table and how data is stored. Each user has a unique ID, and passwords are stored in hashed form to ensure security. This design helps in maintaining data integrity and protecting sensitive information.

Design Advantages

The design of the system offers several advantages:

- Clear separation of layers
- Improved security through structured flow
- Easy to maintain and modify
- Scalable for future enhancements
- Efficient handling of user data

2.3 Implementation

The implementation of this project involves developing a secure web-based authentication system using Python and Flask. The application includes features such as user registration, login, and logout, with proper session management using Flask-Login. User credentials are stored securely in a SQLite database, where passwords are hashed to ensure data protection. The frontend is designed using HTML and CSS to provide a simple and user-friendly interface. All components are integrated to ensure smooth interaction between the user interface, application logic, and database, resulting in a functional and secure system.

2.4 Testing

The system was tested to ensure that all functionalities such as user registration, login, and logout work correctly. Both valid and invalid inputs were checked to verify proper system responses. The application performed as expected, with no major errors, and security features like password hashing and session management were functioning correctly.

Types of Testing

- **Unit Testing**: Individual modules like signup and login were tested separately.
- **Integration Testing**: Checked interaction between frontend, backend, and database.
- **System Testing**: Tested the complete workflow of the application.
- **Security Testing**: Ensured passwords are hashed and unauthorized access is restricted.

2.5Graphical User Interface (GUI) Layout

The GUI of the system is designed to be simple and user-friendly, allowing easy interaction with the application. It includes main pages such as the login page, signup page, and dashboard.

The login page allows users to enter their credentials, while the signup page enables new users to register. The dashboard is accessible after successful login and confirms user authentication.

The interface is developed using HTML and CSS with proper layout and basic styling to ensure clarity and ease of use.

2.6 Customer Testing

Customer testing was conducted to evaluate the usability and functionality of the system. Users interacted with the application by performing tasks such as registration, login, and logout. Feedback was collected regarding ease of use, response time, and overall experience.
The users found the system simple to use and easy to understand. The application performed efficiently, and no major issues were reported during testing.

2.7 Evaluation

The system was evaluated based on its functionality, performance, security, and usability. The evaluation helps to determine how effectively the system meets the project objectives.

- **Functional Evaluation**

The system successfully performs all core functions such as user registration, login, and logout. Each feature works as expected, and the system handles both valid and invalid inputs correctly.

- **Performance Evaluation**

The application shows good performance with fast response time during user interactions. Since the system runs on a local server and uses a lightweight database, operations are executed quickly without noticeable delays.

- **Security Evaluation**

The system implements basic security measures such as password hashing and session management. User credentials are stored securely, and access to protected pages is restricted to authenticated users.

- **Usability Evaluation**

The interface is simple and easy to use. Users can navigate between pages without difficulty, and the system provides clear feedback messages, improving the overall user experience.

- **Limitations**

The system is limited to a local environment and does not include advanced features such as multi-factor authentication or large-scale deployment capabilities.

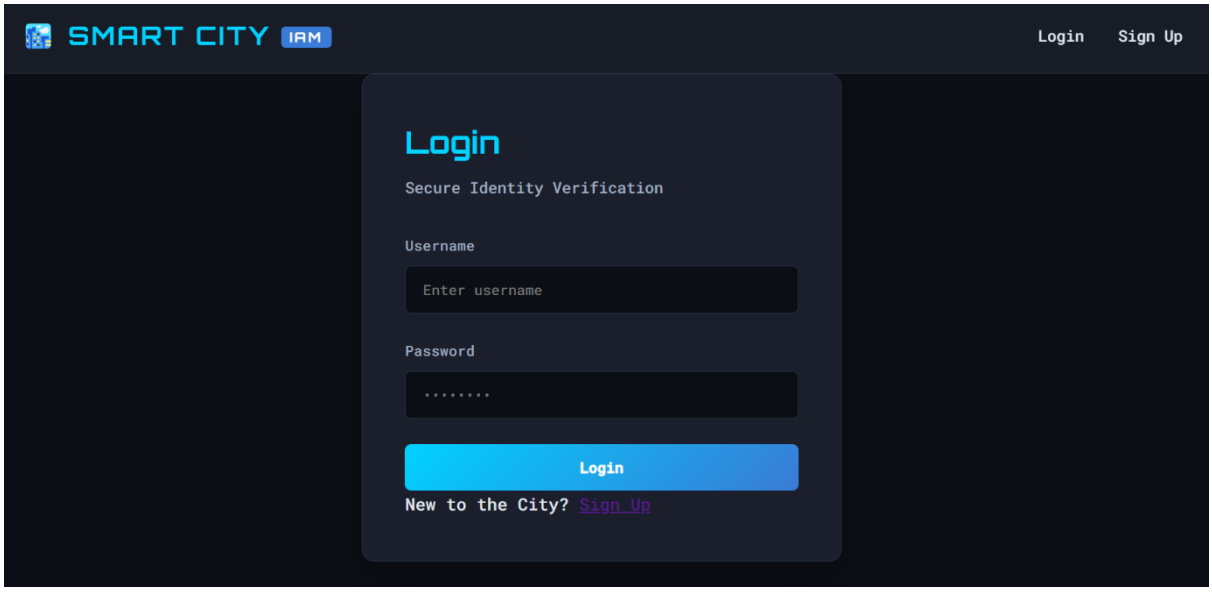
- **Overall Evaluation**

Overall, the system meets the project requirements and provides a secure and efficient authentication mechanism. It is suitable for small-scale applications and serves as a strong foundation for further development.

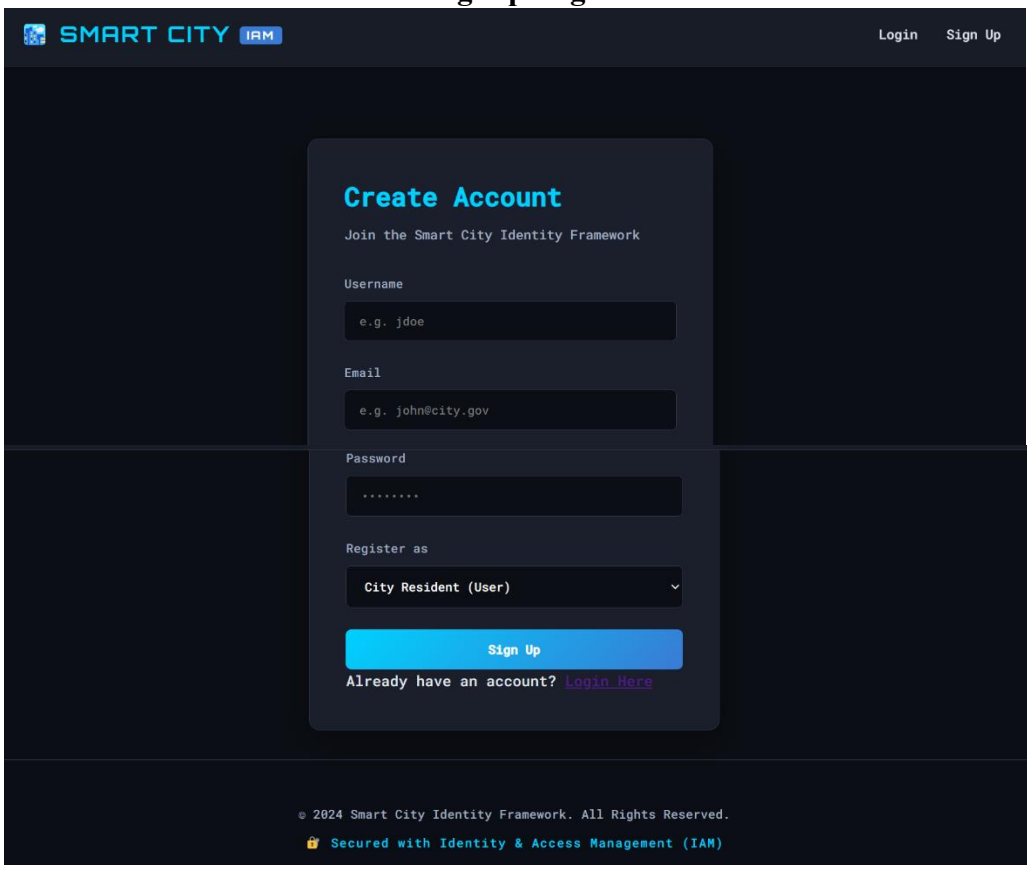
3.SNAPSHOTS OF THE PROJECT

This section includes screenshots of the developed system to demonstrate its functionality and user interface. The snapshots provide a visual understanding of how the application works.

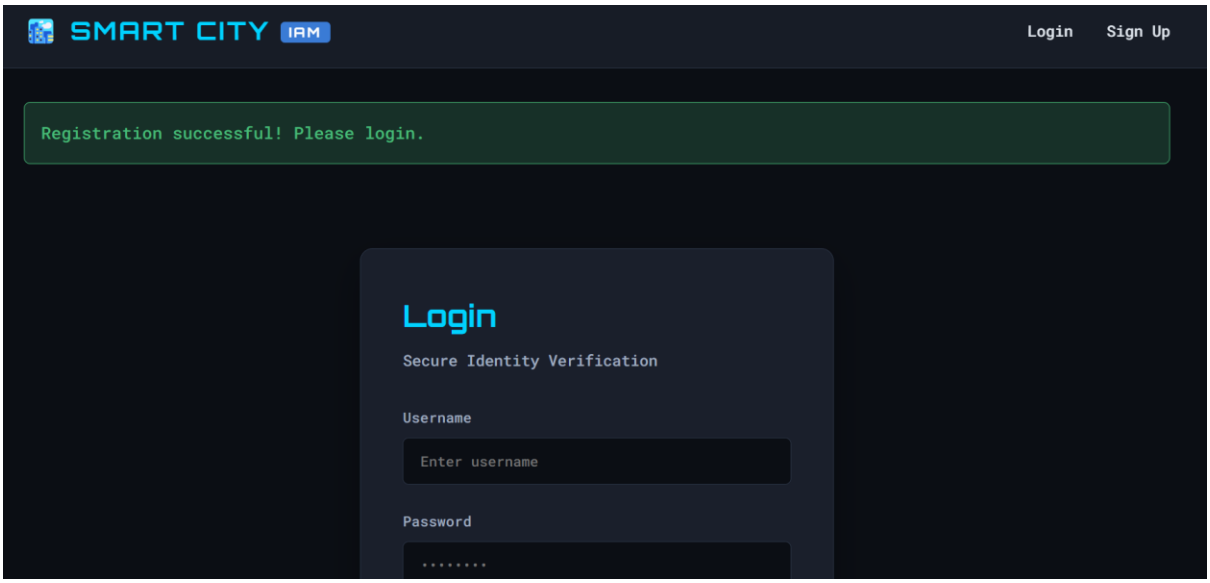
Home Page / Login Page



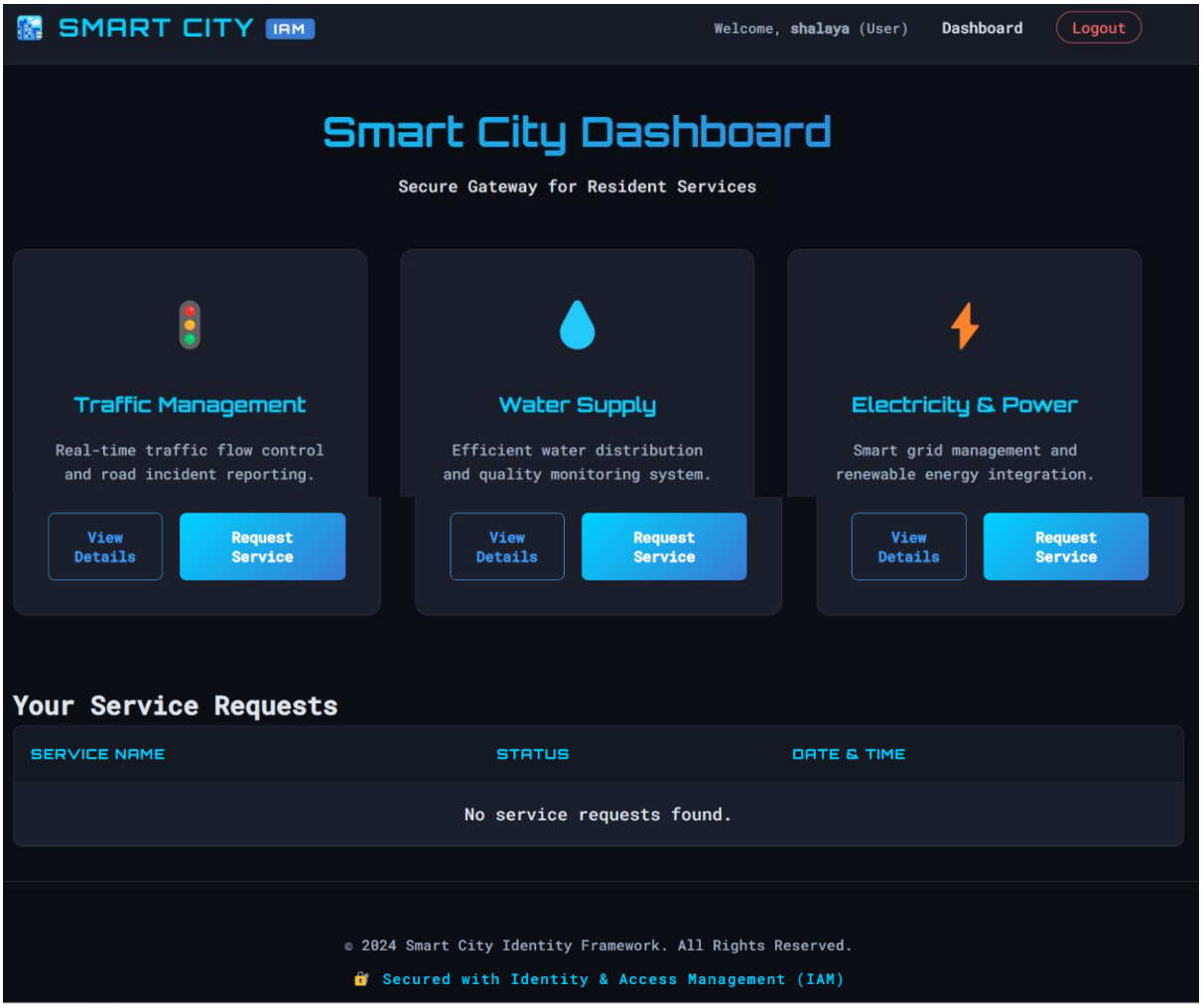
Signup Page



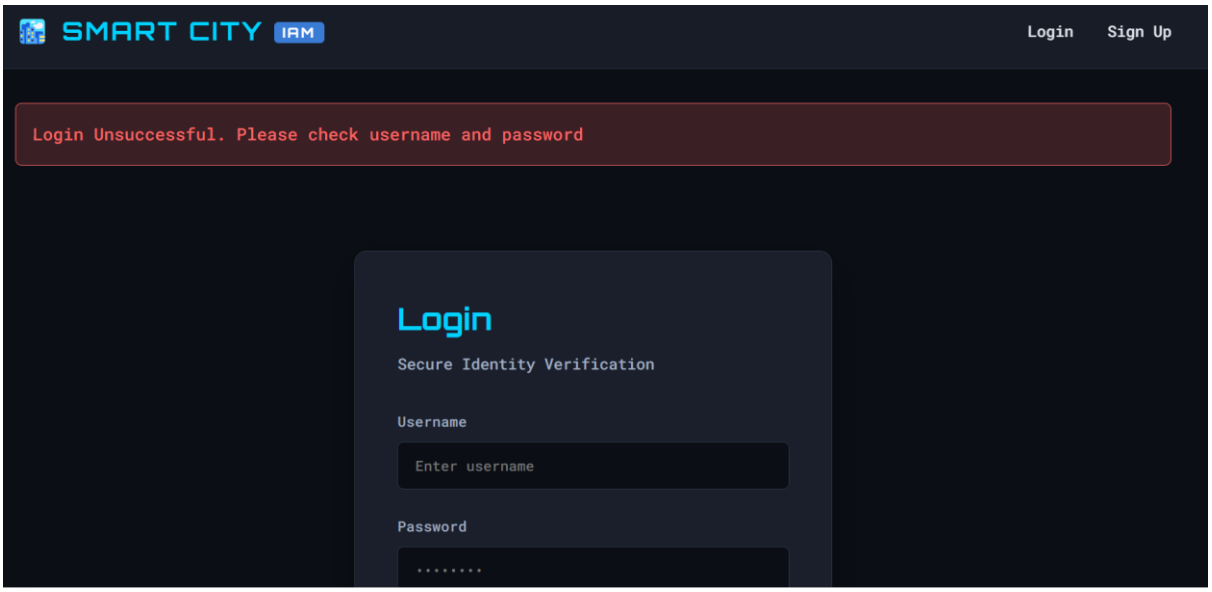
Successful Registration Message



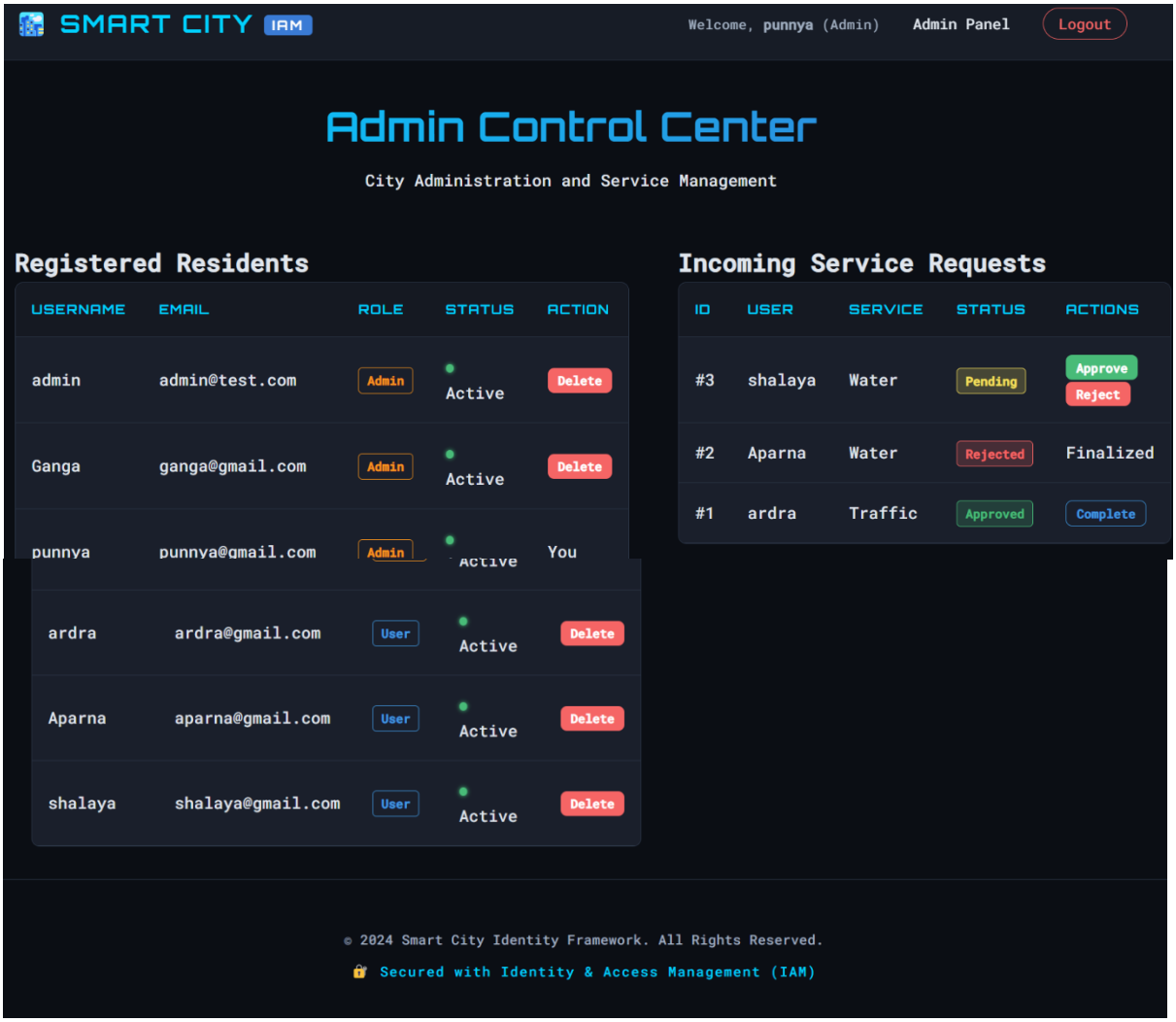
Login Success / Dashboard Page



Error Message (Invalid Login)



Admin Panel



4.CONCLUSIONS

The project "Identity Security Framework for Smart City" focuses on developing a secure and efficient authentication system using Python, Flask, and SQLite. The system successfully implements core functionalities required for user identity management.

Achievement of Objectives

The main objectives of the project, including user registration, login, and secure session management, have been successfully achieved. The system works as expected and provides a reliable authentication mechanism.

System Performance

The application performs efficiently with quick response time and smooth user interaction. It operates well in a local environment and handles user requests without significant delays.

Security Aspects

Basic security measures such as password hashing, input validation, and session management are implemented effectively. These features ensure that user data is protected and unauthorized access is restricted.

Learning Outcomes

This project provided practical knowledge of web application development and cybersecurity concepts such as authentication and authorization. It also helped in understanding how to integrate frontend, backend, and database components.

Final Outcome

Overall, the system meets the project requirements and demonstrates a simple yet secure authentication framework. It serves as a strong foundation for future enhancements and real-world Smart City applications.

5.FURTHER DEVELOPMENT OR RESEARCH

The current system provides a basic and secure authentication mechanism. However, it can be further enhanced by adding advanced features and exploring new technologies to improve performance, security, and scalability.

Multi-Factor Authentication

The system can be improved by adding multi-factor authentication (MFA), such as OTP verification or biometric authentication. This will provide an additional layer of security beyond passwords.

Admin Panel Development

An admin panel can be implemented to manage user accounts, monitor system activities, and control access. This will enhance system management and make it suitable for larger applications.

Role-Based Access Control

The system can be extended to include different user roles such as admin and normal users. This will help in managing permissions and controlling access to different features.

Advanced Database Integration

SQLite can be replaced with more powerful databases like MySQL or PostgreSQL for handling large-scale data and improving performance.

User Interface Enhancement

The GUI can be improved by adding better design, responsiveness, and interactive elements to enhance user experience.

Security Improvements

Additional security measures such as encryption, protection against advanced attacks, and secure API integration can be implemented to strengthen the system.

6.REFERENCES

The following resources were referred to during the development of this project:

- Python Software Foundation, *Python Documentation*. Available at: https://www.python.org/doc/

- Pallets Projects, *Flask Documentation*. Available at: https://flask.palletsprojects.com/

- Pallets Projects, *Flask-Login Documentation*. Available at: https://flask-login.readthedocs.io/

- SQLite, *SQLite Documentation*. Available at: https://www.sqlite.org/docs.html

7.APPENDIX

The appendix section includes additional information and supporting materials related to the project that are useful for understanding the implementation and working of the system.

Sample Code Snippet

The following is a sample of the backend implementation used for user authentication:

```python
from flask import Flask, render_template, request, redirect
from flask_login import LoginManager, login_user, login_required, logout_user
from werkzeug.security import generate_password_hash, check_password_hash

# Example password hashing
hashed_password = generate_password_hash("user_password")
```

Database Schema

The system uses a SQLite database with the following structure:

Field Name	Description
id	Unique user ID
email	User email address
password_hash	Hashed password

Sample Input

- Email: user@example.com
- Password: password123

Sample Output

- Registration Successful
- Login Successful
- Invalid Credentials (Error Message)

Tools Used

- Python
- Flask
- Flask-Login
- SQLite
- Visual Studio Code